

Eric Faltermeyer
 3053092
 Monday 8:00AM to 9:50AM Sumaiya Shomaji
 9/13/2026

File ["/tmp/ipykernel_9324/1095460817.py"](#), line 3

Monday 8:00AM to 9:50AM Sumaiya Shomaji

^
SyntaxError: invalid decimal literal

Next steps: [Explain error](#)

```
import random
import time
import numpy as np
def main():
    while True:

        # gets which task to be calculated by the user
        whichTask = int(input("""Enter the specified number for the wanted task:\n
                                0 for task1A
                                1 for task1B
                                2 for task2
                                3 to exit
                                """))

        # 3 indicates the end of the program
        if whichTask == 3:
            break

        # gets the array from the user
        inputArr = list(map(int, input("Enter the unsorted array separated by spaces: ").split()))

        # calls the according function based on user input
        match whichTask:
            case 0:
                print(taskOneA(inputArr))

            case 1:
                print(taskOneB(inputArr))

            case 2:
                print(taskTwo(inputArr))

        # implements insertion sort with a second "sorted array"
        def taskOneA(inputArr):

            # holds the sorted array
            sortedArr = []

            # gets each number from the input array
            for number in range(len(inputArr)):
                key = inputArr[number]

                # finds where the number belongs
                for i in range(len(sortedArr)):

                    if key <= sortedArr[i]:
                        sortedArr.insert(i, key)
                        break

                # if the number is bigger than everything in sortedArr
                else:
                    sortedArr.append(key)

            # shows sorted array after each iteration
            print(sortedArr)

        return sortedArr

    # implements insertion sort by shifting values in the input array
```

```

def taskOneB(inputArr):

    for sortedIndex in range(1, len(inputArr)):
        key = inputArr[sortedIndex]
        j = sortedIndex - 1

        while j >= 0 and inputArr[j] > key:
            inputArr[j + 1] = inputArr[j]
            j -= 1

        inputArr[j + 1] = key

    # returns the newly sorted input array
    return inputArr

# implements merge sort on an array
def taskTwo(inputArr):

    # splits the array into two equal halves
    middle = len(inputArr) // 2

    # this is the base case of the algorithm
    if len(inputArr) <= 1:
        return inputArr

    # splits the array in two
    leftHalf = inputArr[:middle]
    rightHalf = inputArr[middle:]

    # gets the lengths of the arrays
    leftLen = len(leftHalf)
    rightLen = len(rightHalf)

    # recursively splits the halves until they're ready to be sorted
    if leftLen != 1:
        leftHalf = taskTwo(leftHalf)

    if rightLen != 1:
        rightHalf = taskTwo(rightHalf)

    # creates new indices for the arrays
    i = 0
    j = 0
    sortedArray = []

    # compares the values of the arrays and sorts them correctly
    while i < leftLen and j < rightLen:
        if leftHalf[i] <= rightHalf[j]:
            sortedArray.append(leftHalf[i])
            i += 1
        else:
            sortedArray.append(rightHalf[j])
            j += 1

    if i != leftLen:
        while i != leftLen:
            sortedArray.append(leftHalf[i])
            i += 1
    elif j != rightLen:
        while j != rightLen:
            sortedArray.append(rightHalf[j])
            j += 1

    return sortedArray

main()

# -----
# Experiment
# -----
# 1. Create a large random array
N = 5000 # you can change size (e.g., 10_000)
arr = [random.randint(0, 10**6) for _ in range(N)]
# 2. Time Insertion Sort
arr_copy = arr[:]
start = time.perf_counter()

```

[10663, 23592, 40259, 54529, 56333, 57545, 63131, 69759, 82705, 89874, 91700, 107218, 130973, 139129, 146017, 155381, 156994, 158811, 159694, 161500, 163306, 165112, 166918, 168724, 170530, 172336, 174142, 175948, 177754, 179560, 181366, 183172, 184978, 186784, 188590, 190396, 192202, 194008, 195814, 197620, 199426, 201232, 203038, 204844, 206650, 208456, 210262, 212068, 213874, 215680, 217486, 219292, 221098, 222904, 224710, 226516, 228322, 230128, 231934, 233740, 235546, 237352, 239158, 240964, 242770, 244576, 246382, 248188, 250000, 251806, 253612, 255418, 257224, 259030, 260836, 262642, 264448, 266254, 268060, 269866, 271672, 273478, 275284, 277090, 278896, 280702, 282508, 284314, 286120, 287926, 289732, 291538, 293344, 295150, 296956, 298762, 300568, 302374, 304180, 305986, 307792, 309598, 311404, 313210, 315016, 316822, 318628, 320434, 322240, 324046, 325852, 327658, 329464, 331270, 333076, 334882, 336688, 338494, 340300, 342106, 343912, 345718, 347524, 349330, 351136, 352942, 354748, 356554, 358360, 360166, 361972, 363778, 365584, 367390, 369196, 371002, 372808, 374614, 376420, 378226, 380032, 381838, 383644, 385450, 387256, 389062, 390868, 392674, 394480, 396286, 398092, 399898, 401704, 403510, 405316, 407122, 408928, 410734, 412540, 414346, 416152, 417958, 419764, 421570, 423376, 425182, 426988, 428794, 430600, 432406, 434212, 436018, 437824, 439630, 441436, 443242, 445048, 446854, 448660, 450466, 452272, 454078, 455884, 457690, 459496, 461302, 463108, 464914, 466720, 468526, 470332, 472138, 473944, 475750, 477556, 479362, 481168, 482974, 484780, 486586, 488392, 490198, 492004, 493810, 495616, 497422, 499228, 501034, 502840, 504646, 506452, 508258, 510064, 511870, 513676, 515482, 517288, 519094, 520900, 522706, 524512, 526318, 528124, 529930, 531736, 533542, 535348, 537154, 538960, 540766, 542572, 544378, 546184, 547990, 549796, 551602, 553408, 555214, 557020, 558826, 560632, 562438, 564244, 566050, 567856, 569662, 571468, 573274, 575080, 576886, 578692, 580498, 582304, 584110, 585916, 587722, 589528, 591334, 593140, 594946, 596752, 598558, 600364, 602170, 603976, 605782, 607588, 609394, 611200, 613006, 614812, 616618, 618424, 620230, 622036, 623842, 625648, 627454, 629260, 631066, 632872, 634678, 636484, 638290, 640096, 641902, 643708, 645514, 647320, 649126, 650932, 652738, 654544, 656350, 658156, 660000, 661800, 663600, 665400, 667200, 669000, 670800, 672600, 674400, 676200, 678000, 679800, 681600, 683400, 685200, 687000, 688800, 690600, 692400, 694200, 696000, 697800, 699600, 701400, 703200, 705000, 706800, 708600, 710400, 712200, 714000, 715800, 717600, 719400, 721200, 723000, 724800, 726600, 728400, 730200, 732000, 733800, 735600, 737400, 739200, 741000, 742800, 744600, 746400, 748200, 750000, 751800, 753600, 755400, 757200, 759000, 760800, 762600, 764400, 766200, 768000, 769800, 771600, 773400, 775200, 777000, 778800, 780600, 782400, 784200, 786000, 787800, 789600, 791400, 793200, 795000, 796800, 798600, 800400, 802200, 804000, 805800, 807600, 809400, 811200, 813000, 814800, 816600, 818400, 820200, 822000, 823800, 825600, 827400, 829200, 831000, 832800, 834600, 836400, 838200, 840000, 841800, 843600, 845400, 847200, 849000, 850800, 852600, 854400, 856200, 858000, 859800, 861600, 863400, 865200, 867000, 868800, 870600, 872400, 874200, 876000, 877800, 879600, 881400, 883200, 885000, 886800, 888600, 890400, 892200, 894000, 895800, 897600, 899400, 901200, 903000, 904800, 906600, 908400, 910200, 912000, 913800, 915600, 917400, 919200, 921000, 922800, 924600, 926400, 928200, 930000, 931800, 933600, 935400, 937200, 939000, 940800, 942600, 944400, 946200, 948000, 949800, 951600, 953400, 955200, 957000, 958800, 960600, 962400, 964200, 966000, 967800, 969600, 971400, 973200, 975000, 976800, 978600, 980400, 982200, 984000, 985800, 987600, 989400, 991200, 993000, 994800, 996600, 998400, 1000200, 1002000, 1003800, 1005600, 1007400, 1009200, 1011000, 1012800, 1014600, 1016400, 1018200, 1020000, 1021800, 1023600, 1025400, 1027200, 1029000, 1030800, 1032600, 1034400, 1036200, 1038000, 1039800, 1041600, 1043400, 1045

The time of NumPy's algorithm was much faster than my algorithm's, by orders of magnitude, with my time being about 100 times slower.

